



DEVELOPMENT OF AN LLM-BASED INTELLIGENT AGENT FOR THE KAGGLE LLM 20 QUESTIONS COMPETITION

Course: MCTA4363 Deep Learning

Track: Track 1 – Kaggle Competition

Group members' names: Muhamad Izzudin

Bin Muhamad (2219735)

Muhamad Imran Haziq Bin Haizam (2216429)

Lecturer name: Dr Hassan Firdaus

Overview

- An age-old game where players guess a secret word using minimal information.
- The Goal: Correctly identify the word in 20 rounds or fewer using only "Yes" or "No" questions.
- 2 vs 2 player matchups (Questioner/Answering)
- Platform used is Kaggle Notebook
- Model: Gemma 7B

Gameplay Rules

- **Sequential Round:** Progress turn by turn until 20 rounds
- **Dual Queries:** Questioner must simultaneously submit one deductive question and one guess of the target word.
- **Binary Responses:** Answering agent provides a strictly defined "yes" or "no" reply to shape the next question.
- **Game Limit:** 20 rounds

Baseline Output

```
response="Sure, what's your first question?"
response='the empire state building'
Question 1: Sure, what's your first question?
Answer: no
Guess: the empire state building

response="Here's a question: Is the keyword a living organism?"
response='the answer to the question is: no.'
Question 2: Here's a question: Is the keyword a living organism?
Answer: no
Guess: the answer to the question is: no.

response='Sure, please ask your next question: Is the keyword a planet?'
response='mars'
Question 3: Sure, please ask your next question: Is the keyword a planet?
Answer: no
Guess: mars

response="Here's the next question: Is the keyword a mineral?"
response='the answer is: '
Question 4: Here's the next question: Is the keyword a mineral?
Answer: no
Guess: the answer is:

response='Sure, please ask your next question: Is the keyword a musical instrument?'
response='the answer is: a guitar.'
Question 5: Sure, please ask your next question: Is the keyword a musical instrument?
Answer: no
Guess: the answer is: a guitar.
```

```
response='Does the keyword have a lot of trees?'
response='forest'
Question 1: Does the keyword have a lot of trees?
Answer: no
Guess: forest

response='Does the keyword have a lot of water?'
response='the answer is:'
Question 2: Does the keyword have a lot of water?
Answer: no
Guess: the answer is:

response='Is the keyword a desert?'
response='the answer is: tokyo'
Question 3: Is the keyword a desert?
Answer: no
Guess: the answer is: tokyo

response='Does the keyword have a lot of mountains?'
response='the great wall of china'
Question 4: Does the keyword have a lot of mountains?
Answer: no
Guess: the great wall of china

response='Is the keyword a city?'
response='los angeles'
Question 5: Is the keyword a city?
Answer: no
Guess: los angeles
```


Improvement V1-Better Prompt Instructions

Aspect	Details
What Fixed It	Writing a better system prompt
What Changed in the code	Stopped giving the AI a simple, lazy instruction. Instead, gave it clear rules to think step-by-step, remember the past answers, and stop repeating itself.
The Exact Line That Fixed It	<code>self.formatter.user("Ask one short strategic yes-or-no question that helps narrow down the hidden keyword. Use the previous questions and answers as context. Avoid random, unrelated, or repeated questions...")</code>
Before Example	Prompt: "Please ask a yes-or-no question.": The AI asked completely random, confusing, and repetitive questions.
After Example	The AI asked smart, connected questions to narrow down the answer: "Is it a place?" to "Is it a country?"

V1 Simulation

Improved structured baseline simulation...

Question 1: Is it a place?

Answer: yes

Guess: country

Question 2: Is it a country?

Answer: yes

Guess: tanzania

Question 3: Is it located in Africa?

Answer: yes

Guess: tanzania

Question 4: Is it known for wildlife or safaris?

Answer: yes

Guess: tanzania

Question 5: Is it Tanzania?

Answer: yes

Guess: tanzania

Improvement V2-Giving Examples (Few-Shot)

Aspect	Details
What Fixed It	Giving the AI examples of a good game
What Changed in the code	Put a fake, perfectly played game directly into the AI's memory. This showed the AI exactly how to play step-by-step and how to format its guesses.
The Exact Line That Fixed It	<code>few_shot_examples = ["Is it a place?", "***yes***", "Is it a country?", "***yes***", "Is it located in Africa?", "***yes***", "Is it known for wildlife or safaris?", "***yes***", "***Tanzania***", "Correct!"]</code>
Before Example	The AI made messy, conversational guesses like: "the answer is: tokyo" or "the answer is: a guitar."
After Example	The AI learned how to narrow down options logically and guessed using just one clean word wrapped in stars: <code>***Tanzania**</code> .

V2 Simulation

```
response= Question: Is the hidden keyword a large island nation known for its historical significance and diverse culture?
response='answer:'
Question 1: Question: Is the hidden keyword a large island nation known for its historical significance and diverse culture?
Answer: no
Guess: answer:

response="Sure, here's your question: Is the hidden keyword a city that has a strong maritime presence and is known for its canals and bridges?"
response='question:'
Question 2: Sure, here's your question: Is the hidden keyword a city that has a strong maritime presence and is known for its canals and bridges?
Answer: no
Guess: question:

response="Okay, here's your question: Is the hidden keyword a historical city that's been a hub for trade and commerce throughout the centuries?"
response='sure, guess the keyword:'
Question 3: Okay, here's your question: Is the hidden keyword a historical city that's been a hub for trade and commerce throughout the centuries?
Answer: no
Guess: sure, guess the keyword:

response="The answer is: Is the hidden keyword a historical city that's been a major center for the spice trade?"
response='venice'
Question 4: The answer is: Is the hidden keyword a historical city that's been a major center for the spice trade?
Answer: no
Guess: venice

response='Yes, the answer is: Is the hidden keyword a coastal city known for its aromatic spices and historical significance?'
response='kolkata'
Question 5: Yes, the answer is: Is the hidden keyword a coastal city known for its aromatic spices and historical significance?
Answer: no
```


Improvement V3-Code Guardrails

Aspect	Details
What Fixed It	Using Python code to clean up the AI's text
What Changed in the code	Wrapped the AI inside a smart Python program. If the AI talked too much or added extra words, the code automatically cut out the junk and saved only the real question or guess.
The Exact Line That Fixed It	<pre>1. To Clean Questions: match = re.search(".+?\?", response.replace("'", "")) bad_prefixes = ["question:", "sure, here's your question:", "the answer is:", ...] 2. Backup Plan for Guesses: cleaned = response.lower().replace("guess:", "").replace("the keyword is", "").strip()</pre>
Before Example	The AI outputted extra chit-chat text: "Sure, please ask your next question: Is the keyword a planet?"
After Example	The Python code intercepted the message, wiped away the polite filler words, and left only a perfect question: "Is the keyword a planet?"

V3 Simulation

```
response='Is the keyword a city or a town?'
response='the answer is:'
Question 1: Is the keyword a city or a town?
Answer: no
Guess: the answer is:

response='Are the majority of the inhabitants of the place known to be tall or short?'
response='tanzania'
Question 2: Are the majority of the inhabitants of the place known to be tall or short?
Answer: yes
Guess: tanzania

response='Are the historical buildings in the place mostly made of stone or wooden materials?'
response='tanzania'
Question 3: Are the historical buildings in the place mostly made of stone or wooden materials?
Answer: yes
Guess: tanzania

response='Are the streets of the place mostly straight or winding?'
response='tanzania'
Question 4: Are the streets of the place mostly straight or winding?
Answer: yes
Guess: tanzania

response='Are the rivers that flow through the place mostly perennial or seasonal?'
response='tanzania'
Question 5: Are the rivers that flow through the place mostly perennial or seasonal?
Answer: yes
Guess: tanzania
```


Improvement V4-Advanced code & Rule Guardials

Aspect	Details
What Fixed It	Adding a long list of strict rules and a backup plan in the code
What Changed in the code	• We added a clear checklist of rules that forces the AI to ask broad category questions first and stops it from repeating itself. • We upgraded the Python code to automatically clean the text and rescue the guess if the AI forgets to use the required star formatting.
The Exact Line That Fixed It	<pre>1. The new game rules: "Rules:\n" "- Use previous questions and answers.\n" "- Do not repeat previous questions.\n" "- First identify whether the keyword is a person, place, or thing.\n" "- Ask broad category questions before specific questions.\n" "- Avoid descriptive questions.\n" "- Avoid subjective questions.\n" 2. The backup code for guesses: if guess == "": cleaned = response.lower() cleaned = cleaned.replace("guess:", "").replace("the keyword is", "").strip() guess = cleaned.split(".")[0].strip()</pre>

Aspect	Details
Before Example	The AI might ask the same question twice, ask opinions like "Is it pretty?", or fail its turn if it did not use stars around its guess
After Example	The AI asks basic category questions first ("Is it a person, place, or thing?"). If it types The keyword is Tanzania. without any stars, the backup code fixes it instantly to a clean guess: tanzania.

- **Key Outcome:**
- **Compared with the baseline model, the improved agent demonstrates:**
 - **More structured questioning**
 - **Better use of previous context**
 - **Cleaner output formatting**
 - **Improved keyword extraction robustness**
- **However, occasional irrelevant questions and inconsistent guesses remain open challenges.**

V4 Simulation

```
Initializing Gemma questioner baseline simulation...
```

```
Initializing model
```

```
/opt/conda/lib/python3.10/site-packages/torch/_utils.py:831: UserWarning: TypedStorage is deprecated. It will be removed in the future and UntypedStorage will be the only storage class. This should only matter to you if you are using storages directly. To access UntypedStorage directly, use tensor.untyped_storage() instead of tensor.storage()  
  return self.fget.__get__(instance, owner)()
```

```
response='Is it a place?'
```

```
response='guess the keyword:'
```

```
Question 1: Is it a place?
```

```
Answer: yes
```

```
Guess: guess the keyword:
```

```
response='Ask your question: Is the keyword a person, place, or thing?'
```

```
response='guess the keyword:'
```

```
Question 2: Ask your question: Is the keyword a person, place, or thing?
```

```
Answer: no
```

```
Guess: guess the keyword:
```

```
response='What color is the sky?'
```

```
response='guessing: '
```

```
Question 3: What color is the sky?
```

```
Answer: no
```

```
Guess: guessing:
```

```
response='Is the keyword a place?'
```

```
response='the keyword is: '
```

```
Question 4: Is the keyword a place?
```

```
Answer: yes
```

```
Guess: the keyword is:
```

```
response='Is it a place?'
```

```
response='tanzania'
```

```
Question 5: Is it a place?
```

```
Answer: yes
```

```
Guess: tanzania
```


limitations

- • The Gemma 7B model occasionally generates irrelevant questions that do not contribute to narrowing the search space.
- • The model may repeat previously asked questions despite prompt constraints.
- • Keyword guessing remains inconsistent and may fail to follow the required output format.
- • Performance has not yet reached the efficiency of top leaderboard solutions that use advanced search strategies.
- • Further optimization is required to improve reasoning consistency and question quality.

future works

- • Implement a keyword memory mechanism to prevent repeated questions.
- • Expand few-shot examples with multiple categories (person, place, animal, object) to improve generalization.
- • Introduce information-gain based questioning strategies to maximize search-space reduction.
- • Investigate hybrid approaches combining LLM reasoning with algorithmic search methods inspired by top Kaggle leaderboard agents.
- • Evaluate performance quantitatively using metrics such as average rounds to correct guess, question relevance, and success rate.

**THANK
YOU**
